DAIMON

D M N

An ownerless, DAO-governed deflationary protocol on BNB Chain

No owner · No mint

21 billion supply floor

7-day public timelock

Whitepaper · Draft v0.1

github.com/daimon-dao

Contents

Numbers refer to the page numbering printed in the footer.

1. Abstract

Daimon is a deflationary token governed entirely by its holders, deployed on BNB Chain. It has no owner, no administrator, and no minting function. Its total supply can only decrease — from an initial 1,000 billion tokens toward an immutable floor of 21 billion, below which the code makes further burning mathematically impossible.

Every parameter that governs the protocol — transaction fees, staking mechanics, treasury allocation, even the code itself — can be changed only through an on-chain vote followed by a mandatory seven-day public timelock. No individual, including those who built it, holds the power to alter, pause permanently, accelerate, or bypass that process. This is not a commitment we ask you to trust: it is a property of the deployed contracts, verifiable by anyone at any time.

Holders who lock their tokens receive voting power proportional to both the amount and the duration of their commitment, along with rewards paid in BNB — funded by real trading activity, never by newly issued tokens. There is no inflation, because there is no mechanism capable of producing it.

Daimon is the migration of an existing token toward complete decentralization. Where the previous version had an owner with discretionary control and an 11% transaction fee, the new protocol has no owner, a 4% fee set by community vote, and a hard ceiling written into the code that no future vote can exceed.

The protocol does not promise returns. It guarantees rules: scarcity that cannot be diluted, decisions that cannot be taken in private, and a system that continues to function exactly as specified whether or not anyone is watching over it.

---

2. The Name

In ancient Greece, the *daimon* was not a demon. It was a guiding spirit — a lesser divine force that dwelt in the space between gods and mortals, neither above humanity nor beneath it, but alongside.

The word derives from a verb meaning *to divide*, *to apportion*, *to distribute a share*. The daimon was the one who apportioned destiny: not a ruler who imposed it, but a custodian who accompanied it.

For the Greeks, happiness itself was named after this relationship. *Eudaimonia* — literally, *to have a good daimon beside you*. Flourishing was not something granted from above; it was the result of a good guide, well followed.

The philosophers each understood it differently, and each reading matters here:

Socrates described his *daimonion* as an inner voice that never told him what to do — only when to stop. A restraint, not a command. It intervened solely to prevent error.

Plato, in the Myth of Er, wrote that souls choose their own daimon before birth. The guide is not assigned from above: it is selected by the one who will be guided.

Heraclitus put it most sharply: *character is destiny* — a person’s own disposition is their daimon. Nothing external. The guide and the guided are the same thing.

2.1 The concept came first

This protocol was not built and then named. The name is not a label applied afterwards to a finished system, and the correspondence described below is not a coincidence noticed in retrospect.

The concept of the daimon is what produced the project. Every architectural decision — what the protocol may do, what it must refuse to do, where authority is placed and where it is deliberately absent — was made against that idea. The code is the concept written in Solidity.

The three readings above are not decoration. Each one corresponds to a specific property of the deployed contracts.

2.2 Socrates — the guide that restrains rather than commands

The *daimonion* never told Socrates what to do. It intervened only to stop him. A negative signal: not direction, but restraint.

```
if (block.timestamp < op.readyTimestamp) revert TooEarly();
```

The timelock contains no logic about what constitutes a good decision. It does not approve, advise, weigh or improve. It has no opinion about the proposal passing through it. Its entire contribution is a refusal conditioned on time.

Governance decides. The timelock only prevents anything from happening faster than it can be examined. It is a restraint, and nothing else — which is the only form of authority in the system that operates without a vote.

2.3 Plato — the guide is chosen, before the question arises

In the Myth of Er, souls select their daimon *before* birth. The guide is not assigned by a higher power; it is chosen by the one who will be guided, and chosen in advance.

```
votingPowerAt(voter, proposal.snapshotTimestamp)
```

Voting power is evaluated at the moment a proposal was created, not at the moment of voting. Influence over a decision derives from a commitment made before that decision existed as a question.

The practical effect is that power cannot be acquired in reaction to something. You cannot see a proposal you dislike, buy influence, and use it. The choice precedes the question — which is precisely what Plato described, and precisely what a snapshot enforces.

2.4 Heraclitus — the guide and the guided are the same

Character is destiny. For Heraclitus the daimon was not external at all: a person's own disposition is their daimon. There is no separate entity.

```
GOVERNANCE_ROLE  →  DaimonTimelock
                    (no other holder, in any contract)
```

There is no authority above the protocol. No owner, no administrator, no council, no foundation, no address that can act on the system from outside it. The only role that exists is held by a contract that does nothing except execute what the community has already decided.

What the holders choose is what the protocol becomes. Not because we promise to respect their decisions, but because there is no mechanism through which anyone could fail to. The character of the community is the destiny of the system — and the absence of any other authority is what makes that statement structural rather than aspirational.

2.5 What follows from this

Three properties, three readings, one concept: an authority that apportions rather than accumulates, restrains rather than commands, and is chosen rather than imposed.

Everything in the remainder of this document is an elaboration of that idea into specific mechanisms. Where the two ever conflicted during development, the concept won — including when it made the work considerably harder, as Section 4.5 describes.

---

3. The Problem

3.1 A token with an owner is a promise, not a system

Most tokens are deployed with an administrative account — the *owner* — that retains privileged functions. Depending on the implementation, the owner can change transaction fees, freeze transfers, exclude addresses from limits, withdraw accumulated funds, and in many cases mint new tokens. These capabilities are not hidden: they are written in the contract, visible to anyone who reads it.

The question is not whether such powers exist. It is what stands between those powers and their misuse.

In almost every case, the answer is: the owner's intentions. Nothing else. No procedure, no delay, no requirement to announce, no way for holders to object before the fact. The security model reduces to a sentence that appears, in some form, in thousands of projects: *trust us*.

This is not a criticism of any particular team's honesty. It is an observation about structure. A system whose safety depends on the continued good faith of a single private key is not a safe system — it is a bet on a person. Keys are

lost. Accounts are compromised. People change their minds, face pressure, or make mistakes at three in the morning. And a bet that pays out reliably for two years is still a bet.

3.2 The same structure, at a larger scale

The pattern is not unique to crypto. It is the pattern of the financial system that most people already live inside.

Decisions that determine the value of ordinary people's savings — how much currency to issue, what fees to apply, which terms to change and when — are taken by institutions that do not consult those affected, do not announce changes in advance, and explain them afterward in language engineered to discourage examination. Depositors are not participants; they are the subject matter. The information asymmetry is not a side effect of the system: it is a feature of it.

Inflation is the clearest example. It is not a natural phenomenon. It is a decision, taken by a small number of people, whose effect is to reduce the value of savings held by everyone else. It requires no consent, offers no opt-out, and is rarely described as what it is.

Crypto was supposed to be the alternative. Yet a token whose owner can change the rules at will has simply reproduced the same structure with a smaller budget: an opaque authority, discretionary decisions, and participants who find out afterward.

3.3 What the alternative requires

A genuine alternative cannot rely on better intentions. It has to make the problematic actions structurally impossible.

That means, concretely:

- **No mint function.** Not disabled — absent. Dilution cannot be voted into existence, cannot be enabled by an upgrade, cannot be introduced by anyone under any circumstances, because the code to perform it does not exist.
- **No privileged account.** After deployment, no externally owned wallet holds administrative rights over the protocol. Not the deployer, not the team, not any individual.
- **No silent changes.** Every modification passes through a public proposal, a vote with a minimum participation threshold, and a mandatory delay long enough for anyone to read what is coming and act on it.
- **No unbounded parameters.** Even the community, voting unanimously, cannot raise fees above a ceiling fixed in the code, or burn supply below the floor. The system limits its own governance.
- **No required trust.** Every claim in this document corresponds to code that is public, deployed, and verifiable. Nothing here asks to be believed.

Daimon is an attempt to build exactly this, and to do so for an existing community: not as a new launch, but as the migration of a token that already had an owner toward a system that has none.

3.4 A note on method

This document contains a critique of concentrated financial power. It contains no call to action beyond one: read the code.

There is a form of dissent that consists of demanding that those in charge behave differently. There is another that consists of building something that does not require them to. This project belongs to the second kind. Not because the first is illegitimate, but because a working alternative is a more durable argument than a complaint — and because a system that removes the need for trust cannot be persuaded to change its mind.

The remainder of this document describes how, and — equally important — where the limits are.

---

4. From DMX to DMN

4.1 What existed before

Daimon did not begin as a decentralized protocol. It began as DMX (`0x36EbA94407B53c631eE822C219e94580fadd67c7`), a reflection token on BNB Chain with the structure common to that category: an owner account holding seventeen administrative functions, and an 11% transaction fee.

	DMX (previous)
Reflection fee	4%
Marketing fee	5%
Buyback fee	2%
Total fee	**11%**
Fee ceiling	**none**
Owner	yes, ownership never renounced
Who decides	the owner, with immediate effect
Supply floor	none
Supply reduction	none — see 4.1.2
Staking	none
Governance	none

This section is not written as an accusation. DMX is our own earlier work, and the structure described here was the standard pattern of its generation. It is documented precisely because the community that holds those tokens deserves to know exactly what they are migrating away from — and because several of these properties are not visible without reading the source.

4.1.1 Parameters without bounds

The fee values were not fixed in the code. They were constructor parameters, adjustable afterwards by three setter functions:

```
function setTaxFee(uint256 taxFee) external onlyOwner() {
    _taxFee = taxFee;
}
```

A plain assignment. No `require`, no upper bound, no delay, no announcement. The owner could set any fee to any value, including 100%, effective on the next transaction.

The same applies to the maximum transaction size. Deployed at 0.3% of supply, it currently stands at 0.15% — the owner halved it at some point using a setter with no lower bound. Nothing in the code prevented setting it to one wei, which would have made the token effectively untransferable.

4.1.2 The burn that did not burn

DMX had no burn function. What it had was a buyback that purchased tokens on the market and sent them to the dead address `0x…dEaD`.

The distinction is not semantic. Tokens sent to an unspendable address are economically destroyed but remain in the accounting:

	DMX, on-chain (26 July 2026)
`totalSupply()`	1,000,000,000,000 — unchanged since deployment
Held at dead address	30,058,718,442 (3.0058%)
Circulating	969,941,281,557

The dead address balance continues to grow, since the buyback mechanism is still running; the figure above is a snapshot. The total supply is not a snapshot — it has never decreased by a single wei. Any metric derived from `totalSupply()` — market capitalisation among them — was computed on a figure that overstated the tokens in existence.

This is not unique to DMX; it is how most reflection tokens of that generation handled burning. DMN separates the two steps: tokens are sent to the dead address, and a second permissionless function removes that balance from `totalSupply()` for real, bounded by the floor.

4.1.3 Ownership renunciation, reversible

The contract includes a function worth describing in full, because its appearance and its effect differ:

```
function lock(uint256 time) public virtual onlyOwner {
    _previousOwner = _owner;
    _owner = address(0);              // appears renounced
    _lockTime = block.timestamp + time;
}

function unlock() public virtual {
    require(_previousOwner == msg.sender, ...);
    require(block.timestamp > _lockTime, ...);
    _owner = _previousOwner;          // takes it back
}
```

After calling `lock()`, `owner()` returns the zero address. On any block explorer this is indistinguishable from a permanent renunciation — the signal most commonly cited as proof that a project cannot be altered by its creators. It is temporary, and the previous owner can reclaim control when the timer expires.

This function was never used on DMX (`getUnlockTime()` returns zero, and ownership was never renounced). It is documented here because the pattern is widespread, and because a reader evaluating any token should know that `owner() == 0x0` is not, by itself, evidence of anything.

4.1.4 What DMX did not have — stated fairly

Several powers commonly found in tokens of this kind are absent from DMX, and accuracy requires saying so:

- **No mint function.** None. The supply was fixed at deployment and could not be expanded by anyone. On this point the contract was already sound.
- **No pause and no blacklist.** The owner could not freeze transfers or block specific addresses.
- **No withdraw or rescue function.** The owner could not extract arbitrary funds held by the contract.

The problem with DMX was never that it could be drained. It was that its economics were entirely at the discretion of one account, with no bound, no delay, and no obligation to inform anyone.

One further detail completes the picture: the address receiving the marketing fee is the same address that owns the contract. The party controlling the parameters is also the direct recipient of the revenue they control. Nothing about that arrangement was hidden — it is visible on-chain to anyone who looks — but it is precisely the concentration that the new architecture is designed to eliminate.

4.1.5 On how it came to be this way

DMX was our apprenticeship. We built it with the knowledge we had at the time, following the patterns that were standard for that generation of tokens — patterns shared by hundreds of projects, including many with funded teams and published audits.

What followed was a period of study, and this protocol is what came out of it. The migration exists because the honest response to understanding a limitation is not to defend it.

It is worth noting what DMX did *not* inherit from those templates: no mint function, no pause, no blacklist, no withdrawal mechanism. The four capabilities most often used to extract value from holders were absent from the first version as well. The judgement was there before the vocabulary was.

The new version does not attempt to defend that structure. It replaces it with a better one.

4.2 What replaces it

	DMX (previous)	DMN (new)
Reflection fee	4%	1%
Marketing fee	5%	2% — 60% of which is routed to stakers
Buyback fee	2%	1%
Total fee	**11%**	**4%**
Fee ceiling	**none**	**10%, enforced in code**
Owner	yes, never renounced	**none, structurally**
Who decides	the owner, immediately	on-chain vote + 7-day timelock
Parameter changes	instant, unannounced	13 days minimum, fully public
Minting	not possible	not possible
Supply reduction	tokens moved, supply unchanged	**supply actually reduced**
Supply floor	none	**21B, immutable**
Exclusion from reflection	owner-toggleable at runtime	**immutable set, dead address only**
Swap slippage protection	none — accepts any output	bounded, with quote from router
Marketing recipient	same address as the owner	governance-set, expected to be a multisig
Staking	none	vote-escrow, rewards in BNB
Governance	none	full on-chain governance

Two points deserve emphasis.

The fee reduction is not the important part. Fees can be adjusted by any project at any time; a lower number proves nothing about how it was arrived at. What matters is that DMN's 4% was set by an on-chain vote, executed after a seven-day public timelock, and that no single account can change it again. The full record of that decision — proposal, vote, delay, execution, with transaction hashes — is documented in Section 8.

The ceiling matters more than the current value. The code rejects any total fee above 10%. This is not a policy that governance can revise: it is a `require` statement in the contract. Even if every token holder voted in favour, the transaction would revert. The practical consequence is that DMN can never become as expensive to transact as DMX is today, regardless of who controls the protocol in ten years.

The row on supply reduction is the least obvious and the most consequential. In DMX, buyback tokens accumulated at the dead address while `totalSupply()` stayed at its deployment value forever. In DMN, a permissionless function removes that balance from the total for real — down to the floor and no further. The reported supply is the supply.

4.3 How the migration works

The migration is a 1:1 exchange with no fee and no deadline pressure beyond a fixed window.

At deployment, the entire initial supply of 1,000 billion DMN is created inside the migration contract. Not in a team wallet, not in a treasury, not in a distribution contract — inside the contract that exists solely to exchange old tokens for new ones. No allocation is reserved for founders, advisors, or private investors, because no such allocation exists to reserve.

The process for a holder is three steps:

1. **Approve** — authorise the migration contract to receive a specified amount of legacy tokens.
2. **Claim** — the contract takes the legacy tokens and sends back exactly the same number of DMN. No fee is applied in either direction.
3. **Done** — the legacy tokens are transferred to the DAO treasury, not destroyed.

Every claim is a transfer of tokens that already existed. Nothing is minted. The total supply of DMN does not change by a single wei during the entire migration, and this can be verified on-chain at any point.

4.4 What happens to tokens that are never claimed

Not every holder will migrate. Wallets are abandoned, keys are lost, announcements go unread. Some portion of the supply will remain in the migration contract when the window closes.

Those tokens are not burned automatically, and they do not become anyone's property. They remain locked in the migration contract until the DAO decides otherwise — and that decision requires the full governance cycle: a proposal, a vote, a seven-day timelock, and an execution that anyone can trigger.

The function that performs this transfer, `sweepUnclaimed()`, has three constraints written into it:

- it can only be called by the timelock, meaning only as the result of a passed vote;
- it reverts if called before the migration deadline has passed;
- it can send the tokens to one address only — the treasury — whose address is `immutable`, fixed at deployment and unchangeable by anyone, including governance.

There is a detail worth stating plainly, because it will otherwise be discovered and questioned: the migration contract is a token holder like any other, and therefore accumulates reflection from ordinary transfers during the migration window. The balance swept to the treasury will be slightly larger than the amount left unclaimed. This is the reflection mechanism working as designed, and the surplus follows the same path as the rest — to the DAO, decided by vote.

4.5 Why a migration rather than renouncing ownership

A reasonable question: if the goal was to remove the owner, why not simply call `renounceOwnership()` on the existing contract?

Because renouncing ownership on DMX would have produced a token that nobody could change — including a token that nobody could fix, improve, or govern. The fee would have been frozen at 11% permanently. There would be no staking, no voting, no way for holders to decide anything. The result is not decentralization; it is abandonment.

Decentralization means decisions are taken collectively, not that decisions become impossible. That requires a governance system, a timelock, a staking mechanism to weight participation, and contracts designed around the absence of an administrator from the first line. None of that could be retrofitted onto the existing contract.

The migration is the cost of doing it properly.

---

5. Architecture

5.1 Five contracts

The protocol consists of five contracts, each with a narrow responsibility.

DaimonV2 — the token. Implements the ERC-20 interface with reflection-based fee redistribution, the buyback and burn cycle, and the immutable supply floor. Deployed behind a UUPS proxy, upgradeable only through governance.

DaimonStaking — vote-escrow staking. Holds locked tokens, computes voting power as a function of amount and lock duration, maintains historical checkpoints of that voting power, and distributes BNB rewards proportionally.

DaimonGovernor — the governance engine. Accepts proposals from holders above a threshold, runs the voting period, evaluates quorum against a snapshot taken at proposal creation, and queues successful proposals into the timelock.

DaimonTimelock — the mandatory delay. Holds every approved decision for seven days before it can be executed, and is the only account with authority to call privileged functions on the other contracts. It administers itself: no external account can modify its parameters or its roles.

DaimonMigration — the 1:1 exchange. Holds the initial supply, converts legacy tokens on demand, and can transfer any unclaimed remainder to the treasury after the deadline, on instruction from the timelock only.

5.2 Where authority sits

The distribution of authority is the property most worth verifying independently, so it is stated here in full.

No contract has an owner. There is no `Ownable` inheritance, no `owner()`, no `onlyOwner` modifier anywhere in the protocol. Access control is role-based, and after deployment exactly one role holder exists:

```
GOVERNANCE_ROLE  →  held by DaimonTimelock, and nothing else
```

The deployer holds no role. This is not a matter of intention: the deployment script renounces every role it temporarily holds during setup, and then asserts — with fourteen separate checks that abort the deployment if any of them fails — that no externally owned account retains authority over any contract. If a single check fails, no deployment occurs.

Consequently, changing the fee, changing the marketing wallet, changing the staking contract, adding a lock option, sweeping unclaimed migration tokens, or upgrading the token implementation are all operations that can only occur at the end of this sequence:

```
proposal → 1-day delay → 5-day vote → quorum reached
         → queue → 7-day timelock → execution
```

Execution itself is permissionless: once the timelock has elapsed, any address can trigger it. The decision was made by the vote; the execution is mechanical.

5.3 The two exceptions

Two elements sit outside this structure, and both are deliberate.

Immutable addresses. The dead address (destination of burned tokens) and the migration treasury are fixed at deployment and cannot be changed by anyone — not by the deployer, not by governance, not by an upgrade. Some destinations should not be redirectable, even by a majority.

The guardian. One account holds one power: it can pause the contract in an emergency. It cannot change fees, move funds, alter governance, mint, or upgrade anything. It exists because in the early life of a protocol, seven days of timelock is too slow a response to an exploit in progress.

The guardian’s authority expires 36 months after deployment. This is not a commitment to renounce it; it is a timestamp comparison in the code that no one can remove or postpone. After that date the pause function stops working permanently — while the unpause function continues to work, so that no one can leave the protocol frozen.

Section 8.6 describes the guardian’s constraints in full.

---

6. Tokenomics

6.1 Supply

The initial supply is 1,000,000,000,000 DMN — one thousand billion tokens, with 18 decimals. This number was fixed at deployment and can never increase.

The reason it can never increase is not a policy. **There is no mint function in the contract.** Not disabled, not permission-gated, not restricted to governance: absent. No sequence of votes, no upgrade, no compromised key, and no future decision can produce a single additional token, because the code capable of producing one does not exist.

Supply can only move in one direction: down.

6.2 The floor

Burning is bounded. The contract defines an immutable minimum supply of 21,000,000,000 DMN — twenty-one billion — below which burning is mathematically impossible.

```
INITIAL SUPPLY   1,000,000,000,000 DMN
                          |
                          ▼  (burn, one direction only)
                          |
FLOOR              21,000,000,000 DMN   ← cannot be crossed
```

The burn function checks this bound before executing and stops exactly at the floor: if a burn would take supply below 21 billion, only the portion that reaches the floor is burned, and the remainder is not. Called again afterwards, the function performs no operation.

This ceiling on deflation exists for a practical reason. A supply that can approach zero eventually breaks: divisibility limits, rounding behaviour and liquidity all degrade as the token count collapses. The floor is the point at which the

protocol stops shrinking and starts operating in a different mode — described in 6.6.

6.3 Transaction fees

Every transfer applies a fee, currently 4% of the transferred amount, split into three components:

Component	Rate	Destination
Reflection	1%	redistributed to all holders
Buyback & burn	1%	accumulated, then used to buy and burn
Marketing / operations	2%	60% to staker rewards, 40% to operations
Total	**4%**	

Two structural properties matter more than the current numbers.

The ceiling. The contract rejects any configuration where the total fee exceeds 10%. This is enforced by a `require` statement, not by policy. A proposal setting fees to 11% would pass a vote, wait out the timelock, and then revert on execution. Governance is bounded by the code it governs.

Who can change them. Only the timelock, meaning only the outcome of a completed governance cycle. The current 4% is itself the result of one: the first proposal in Daimon's history reduced fees from 5% to 4%, and its full record appears in Section 8.

6.4 Reflection

The reflection component redistributes 1% of every transfer to all holders, proportionally to their balance, without any transaction being required to claim it.

Mechanically, balances are stored in an internal unit of account rather than in tokens directly. Each taxed transfer reduces the total of that internal unit, which increases the number of tokens each unit represents. Every holder's balance grows without any transfer occurring — the redistribution is a change in the conversion rate, applied simultaneously to everyone.

The practical consequence is that holding tokens produces a slow accumulation funded by trading activity, with no action required and no gas paid.

One address is excluded from reflection: the dead address that receives burned tokens. Excluding it prevents burned supply from accruing redistribution that would inflate the apparent burn figure. The tokens counted as burned are burned; nothing is added to that number by the mechanism itself.

There are no other exclusions, and no function exists to create one. This is a deliberate structural choice: the ability to toggle addresses in and out of reflection at runtime has historically been the source of the most serious accounting failures in reflection tokens. That entire class of failure is absent here because the capability is absent.

6.5 The burn cycle

Burning is not scheduled, announced, or triggered by anyone in particular. It is a consequence of trading.

```
1.  Trading occurs → fees accumulate as DMN inside the contract

2.  Once accumulated fees exceed a threshold, and a sale to the
    liquidity pool occurs, the contract sells the accumulated
    tokens on the market → receives BNB

3.  The BNB is split:
        marketing share  →  60% to the staking reward pool
                            40% to operations
        buyback share    →  retained in the contract

4.  When the retained BNB exceeds a threshold, the contract buys
    DMN on the open market and sends it to the dead address

5.  burnDeadBalanceToFloor() removes that balance from total
    supply — permanently, and down to the floor at most
```

Step 5 is **permissionless**: any address can call it, at any time, with no special role required. No one can prevent it, and no one can force it past the floor. The function has no privileged caller because it needs none — its only possible effect is to reduce supply toward a bound fixed in the code.

Steps 2 and 4 are automatic and use the public liquidity pool. The swaps are protected by a slippage bound and

wrapped so that a failed swap cannot block transfers: if market conditions make a swap unfavourable, it is skipped and retried later rather than reverting the user's transaction.

6.6 What happens at the floor

When total supply reaches 21 billion, burning stops permanently. It does not resume, and no vote can restart it.

At that point the buyback component no longer has a destination that reduces supply. The protocol's design specifies that the revenue previously directed to burning is redirected in full to stakers — the mechanism switches from reducing supply to distributing yield, automatically, based on a supply check rather than a decision.

This is a distant scenario. It is described here because a protocol should specify its terminal state, not because it is imminent.

---

7. Staking

7.1 Lock and weight

Staking in Daimon is vote-escrow: tokens are locked for a chosen duration, and both voting power and reward share are weighted by that duration.

Lock period	Multiplier
30 days	1×
90 days	1.5×
180 days	2.2×
365 days	4×

Voting power is the staked amount multiplied by the period's weight. One million DMN locked for a year carries the same weight as four million locked for a month.

The intent is explicit: **influence should track commitment, not only capital.** A holder who accepts a year of illiquidity has demonstrably more at stake in the protocol's future than one who can exit in thirty days, and the weighting reflects that.

Locked tokens cannot be withdrawn before the lock expires. There is no early exit, no penalty option, no administrative override. The commitment is the mechanism; making it reversible would remove its meaning.

7.2 Rewards in BNB

Staking rewards are paid in **BNB**, never in DMN.

This is a consequence of the supply design rather than a preference. With no mint function, rewards denominated in DMN could only come from a pre-allocated reserve — which would eventually be exhausted, and which would have required holding a large team-controlled balance from day one. Neither was acceptable.

Rewards therefore come from the marketing share of transaction fees: 60% of that share is converted to BNB by the protocol and deposited into the staking reward pool, where it is distributed among stakers in proportion to voting power.

Three properties follow:

- **No inflation.** No token is created to pay anyone. The reward pool is funded by trading activity that has already occurred.
- **No sell pressure.** A staker claiming rewards receives BNB, not DMN. Claiming never puts tokens on the market.
- **External value.** Rewards are denominated in the chain's native asset, whose value does not depend on Daimon.

The combined effect is that staking removes tokens from circulation while returning value that does not have to be sold back into the same market.

Rewards accrue continuously and are claimed on demand. If rewards arrive at a moment when no tokens are staked, they are held and distributed to the next stakers rather than lost.

7.3 Voting power checkpoints

The staking contract does not only record current voting power. It records its history.

Every change — a new lock, a withdrawal — writes a checkpoint with a timestamp. This allows the governance contract to ask a specific question: *what was this address's voting power at the moment proposal N was created?*

This is what prevents the most obvious governance attack. Without historical checkpoints, an actor could observe a proposal they dislike, acquire and stake a large position, and vote it down with power purchased after the question was raised. With them, the vote is settled against a snapshot taken at proposal creation: power acquired afterwards counts for nothing.

We tested this specifically, with a simulated attacker holding an overwhelming position staked immediately after a proposal's creation. Their voting power at the snapshot was zero, and the vote was rejected — as designed.

7.4 A design choice: voting power does not decay

In the vote-escrow model introduced by Curve Finance, voting power decays linearly as the lock approaches expiry. A four-year lock confers full weight on day one and zero weight at maturity; holding influence requires continuously re-locking.

Daimon does not implement decay. Voting power is fixed at the moment of staking and remains constant until withdrawal, including after the lock has expired.

This is a deliberate divergence, and it has a real consequence worth stating openly: a holder who locks at 4×, waits out the year, and never withdraws retains full voting weight indefinitely while holding the ability to exit at any time. Influence can concentrate among early, long-term stakers rather than tracking present commitment.

We accept this trade-off for three reasons. It is simpler and predictable — voting power is a number a holder can understand without recomputing it every block. It rewards demonstrated loyalty rather than requiring perpetual re-commitment. And it avoids the friction of a mechanism that penalises holders precisely as they approach the end of a commitment they honoured.

A decaying model is not ruled out permanently. It would be a significant redesign, and if the community concludes it is preferable, it can be introduced through the same governance process that governs everything else. The choice is documented here rather than discovered later.

7.5 Limits

Two constraints apply to staking and are stated for completeness.

A single stake transaction is bounded by the token's maximum transaction size (0.5% of supply), an anti-dump limit that applies to all transfers. Very large positions must be staked across multiple transactions.

The staking contract is excluded from transaction fees automatically when it is registered, so staking and unstaking do not incur the 4% fee. This exclusion applies to the contract itself, not to any individual, and is set by the same governance-only function that registers the contract.

---

8. Governance

8.1 The cycle

Every change to the protocol follows the same path. There are no shortcuts, no emergency procedures that bypass it, and no accounts exempt from it.

```
PROPOSAL          any holder above the proposal threshold
     │            submits a target contract and an encoded call
     ▼
1-DAY DELAY       nothing can be voted immediately
     │            voting power is snapshotted at proposal creation
     ▼
5-DAY VOTE        For / Against / Abstain, weighted by voting power
     │            held at the snapshot
     ▼
QUORUM CHECK      at least 10% of total voting power must participate
     │            (For + Abstain — see 8.3)
     ▼
QUEUE             the approved call enters the timelock
     ▼
7-DAY TIMELOCK    mandatory public delay, 604,800 seconds exactly
     │            no one can shorten it
     ▼
EXECUTION         permissionless — any address can trigger it
```

Minimum elapsed time from proposal to effect: **thirteen days**. This is not a convenience. It is the window in which anyone — a holder, a researcher, a journalist, an adversary — can read what is coming and act on it before it happens.

8.2 Voting power at the snapshot

Votes are weighted by voting power held **at the moment the proposal was created**, not at the moment of voting.

This single property closes the most direct attack on any token-weighted governance: observing a proposal, acquiring a large position, and voting with influence purchased specifically to decide that question. Under snapshot voting, tokens staked after a proposal exists carry zero weight on it.

The mechanism is a historical checkpoint written on every change to a holder’s staked position. The governance contract queries that history rather than the present state.

8.3 What counts toward quorum

Quorum is measured as the sum of **For and Abstain** votes, against 10% of total voting power at the snapshot. Against votes are counted in determining the outcome, but not in reaching quorum.

This distinction is not cosmetic, and it was not in our original implementation. During adversarial testing we found that counting Against votes toward quorum produced a perverse incentive: a minority opposing a proposal could, by voting against it, supply the participation needed to validate the vote — and thereby cause the proposal to pass. Remaining silent was strictly more effective than voting no.

We reproduced the scenario in a test, confirmed it, and changed the calculation to match the OpenZeppelin standard, which excludes Against votes from quorum for exactly this reason. The finding, the fix and the test that demonstrates the corrected behaviour are all in the public repository.

A proposal that nobody supports now fails by not reaching quorum — which is the correct outcome, and requires no one to actively defeat it.

8.4 Bounds on governance itself

A system where the majority can do anything is not safer than a system where one person can do anything; it is only slower. Several constraints are therefore placed above governance, in the code, where no vote can reach them:

Constraint	Value	Who can change it
Maximum total fee	10%	**no one**
Minimum supply (burn floor)	21 billion	**no one**
Minimum quorum	10%	**no one**
Minimum timelock delay	7 days	**no one**
Dead address	fixed at deployment	**no one**
Migration treasury address	fixed at deployment	**no one**
Mint capability	does not exist	**no one**

A proposal violating any of these would pass a vote, wait out the timelock, and revert on execution. The protocol declines instructions it was built to refuse.

8.5 What governance can do

Within those bounds, the DAO controls the protocol completely:

- adjust the fee split and total (up to the ceiling)
- change the marketing and operations recipient
- change the staking contract and reward split
- add or disable staking lock options
- adjust operational thresholds (swap trigger, slippage tolerance, maximum transaction size)
- sweep unclaimed migration tokens to the treasury after the deadline
- **upgrade the token implementation itself**

The last item deserves emphasis. The token is deployed behind a UUPS proxy, and the upgrade authorisation function is restricted to the timelock. This means the protocol can evolve — but only through the full public cycle, and never at the discretion of any individual. It also means the community carries real responsibility: an upgrade is the most powerful instrument in the system, and the seven-day delay exists so that a bad one can be seen coming.

8.6 The guardian

One account holds one power outside the governance cycle. The guardian can **pause the contract**. That is the entire scope of its authority.

It cannot change fees, move funds, alter governance parameters, mint, upgrade, or influence a vote. It can stop transfers, and nothing else.

It exists for a narrow reason: in the early life of a protocol, seven days of timelock is not a viable response time to an exploit in progress. The guardian is the fire alarm, not a seat at the table.

Three constraints define it:

It expires. The guardian's authority ends 36 months after deployment. This is a timestamp comparison in the code, not a commitment — no one, including governance, can remove or extend it. After that date the pause function stops working permanently.

It cannot trap the protocol. The unpause function continues to work after the guardian's expiry. The power to freeze disappears; the power to resume does not. No one can leave Daimon paused indefinitely.

It is visible. Pausing is an on-chain event. There is no way to use this power quietly.

The guardian is a temporary compromise with reality, bounded in scope, bounded in time, and designed to disappear without requiring anyone's cooperation.

---

9. Case study: Proposal #0

Every claim in Section 8 describes intended behaviour. This section describes what actually happened when the system was used.

The following is the complete record of the first decision taken by the Daimon DAO, executed on BSC testnet. Every step is a public transaction.

9.1 The decision

Proposal #0 — reduce total fees from 5% to 4% (reflection 1%, buyback 1%, marketing 2%).

The proposal was created by a holder, targeted the token contract, and encoded a call to `setFees(10, 10, 20)`. Nothing about its creation required permission.

Its on-chain description reads:

> *"Riduzione fee totale al 4% (tax 1%, buyback 1%, marketing 2%)"*

The text is in Italian and appears untranslated throughout the interface, including in the English version. Proposal descriptions are on-chain content authored by the proposer and immutable once submitted; translating them in the interface would break the correspondence between what a user reads and what the blockchain contains. What is displayed is byte-for-byte what was recorded.

9.2 The timeline

All times UTC. Every value below is read from contract state, not from project documentation.

Stage	Timestamp	UTC
Created — voting power snapshotted	`1783467501`	7 Jul 2026, 23:38:21
Voting opened (after 1-day delay)	`1783553901`	8 Jul 2026, 23:38:21
Vote cast — 3,000,000 in favour		9 Jul 2026, 22:36:42
Voting closed (after 5-day period)	`1783985901`	13 Jul 2026, 23:38:21
Queued into the timelock	`1783991100`	14 Jul 2026, 01:05:00
Executable from	`1784595900`	**21 Jul 2026, 01:05:00**
Executed		**21 Jul 2026, 01:11:09**

Result: 3,000,000 voting power in favour, 0 against, 0 abstaining, against a required quorum of 300,000. Final status: **Executed**.

The interval between queueing and executability is 604,800 seconds — seven days, to the second.

Transactions:

Action	Hash
Propose	`0xa6e465fb70da2b587f8ab7795a22cfc7c29bc984d571020260178b6af2cb5035`
Vote	`0x90e22db141f83b1b9fb004faaabde852da721b55856594122c1f0cf9564e1480`
Queue	`0x3d5adeac84a205edd963af483de98bd0f40be0491532ed4dc9b0f2418fee2920`
Execute	`0x5aa519a9884d24037f0cb903f3565f1a9e5e87529e5d4c1baa3f0c054302fe5f`

The complete cycle is public. Nothing about this decision happened off-chain.

All four were sent from the same address. The propose, vote and queue transactions occupy nonces 6, 8 and 9 in that wallet's sequence — the order the protocol requires, with the vote correctly positioned between the other two.

9.3 The part that matters most

On 17 July — four days before execution was possible — we attempted to execute the proposal.

It was a human error: a mistake about the date, made by people who had built the system and knew precisely how it worked. We ran a coherence check before signing, and it returned the reason the attempt would fail:

```
readyTimestamp   21 Jul 2026, 01:05:00 UTC
current time     17 Jul 2026, 08:34:44 UTC
remaining        318,613 seconds  (3 days, 16.5 hours)
```

Had we proceeded anyway, the transaction would have reverted with `TooEarly`. The contract does not distinguish between a malicious early execution and a confused one; it refuses both identically.

This is the entire argument for a timelock, demonstrated rather than asserted. It does not exist because we distrust bad intentions. It exists because good intentions are not sufficient — people lose track of dates, work at three in the morning, and are certain about things that are wrong. The system is not.

Four days later, execution succeeded six minutes after the window opened. Not one second earlier, because it was not possible.

9.4 Verification of the result

The fee change was confirmed on two independent levels.

Contract state, read directly:

```
taxFee        = 10   →  1%   (reflection)
buybackFee    = 10   →  1%   (buyback & burn)
marketingFee  = 20   →  2%   (marketing / operations)
                        ──
                        4%
```

Economic behaviour, measured on a real transfer of 1,000,000 DMN:

	Amount
Sent	1,000,000.000000 DMN
Fee applied	**40,000.000000 DMN — exactly 4.00%**
Received	960,000.000000 DMN
Total supply change	zero

Before execution the same transfer would have carried a 50,000 DMN fee. The governance cycle did not merely update a stored value: it changed how the token behaves, and the change is measurable to the wei.

9.5 The proposal cannot be executed twice

Calling `execute(0)` again returns:

```
execution reverted: 0x0dc10197   →   AlreadyExecuted()
```

The check is the first statement in the function, before any other logic. A signed and paid transaction would fail identically. The interface offers no button for it, because the executed state has no available actions.

9.6 A proposal that failed

Proposal #1 was created deliberately as a test and deliberately not voted on by anyone.

At the end of its five-day voting period it reached zero votes, failed to meet quorum, and its status became **Defeated** — automatically, with no transaction required and no one acting to reject it.

This is the outcome we wanted to observe on-chain. In a system with no veto power, a proposal without support does not need to be stopped. It expires.

9.7 What this record demonstrates

Between these two proposals, every state of the governance machine has been observed with real transactions: Pending, Active, Succeeded, Queued, Executed, and Defeated. The rejection paths — execution before the timelock expires, and execution of an already-executed proposal — were both triggered and both refused.

The complete record, including transaction hashes for every step, is published in `TESTNET_RESULTS.md` in the project repository.

---

10. Security

10.1 Threat model

Security begins by naming the adversaries. The following actors are considered, with what each can and cannot do.

An external attacker. Can call any public function, in any order, at any time, with any amount. Cannot acquire an administrative role, because none is assignable: `GOVERNANCE_ROLE` administers itself and there is no `DEFAULT_ADMIN_ROLE` through which it could be granted.

A whale. Can accumulate a large position and stake it for maximum weight. Cannot vote on any proposal created before their stake, because voting power is evaluated against a historical snapshot. Cannot exceed the maximum transaction size in a single transfer.

Governance itself. Can change every operational parameter and upgrade the token implementation. Cannot mint, cannot burn below the floor, cannot raise fees above 10%, cannot lower the quorum below 10% or the timelock below seven days, cannot redirect the dead address or the migration treasury. The protocol constrains its own governance.

The guardian. Can pause the contract. Cannot do anything else, and loses even that after 36 months.

The deployer. Can deploy the contracts and pay the gas. Holds no role afterwards: the deployment script renounces every temporary permission and then verifies, with fourteen assertions that abort deployment on failure, that no externally owned account retains authority anywhere in the system.

The full threat model, including the reasoning behind each conclusion, is published as `THREAT_MODEL.md` in the repository.

10.2 What has been tested

The protocol carries 74 automated tests, all passing, across five categories.

Unit tests cover individual functions and their boundaries.

Governance sequence tests verify that no ordering of operations produces an execution without a passed vote, a completed queue and an elapsed timelock.

Fuzz tests run each property against 512 randomised inputs: that transfers never create tokens, that staking grants exactly the weighted voting power, that withdrawal returns capital 1:1, that migration is exactly 1:1, that rewards never exceed the BNB deposited, that fees can never be set above the cap.

Invariant tests are handler-based: a driver hammers the system with random sequences of actions — 16,384 calls per invariant — while the following must hold at every step:

- total supply never exceeds the initial supply and never falls below the floor
- total voting power equals the sum of active locks, and equals the sum of per-user voting power
- migration accounting is conserved
- the reward balance equals deposits minus claims
- no administrative role is acquirable by any actor

Adversarial tests target the system as an attacker would: snapshot manipulation, extreme boundary values (one wei, the entire supply, the exact floor, one second before the timelock expires), and game-theoretic incentives.

Static analysis is run with Slither. Every High and Medium finding has been examined and documented — with the reasoning — as either a false positive in this context or an already-mitigated condition; the classification is published in `THREAT_MODEL.md`. Low-severity hardening suggestions were applied to the code.

10.3 What we found ourselves

Two findings emerged from our own adversarial round, after the code was otherwise complete. Both are documented here because a security section that reports only successes is not a security section.

Finding 1 — Against votes counted toward quorum. As described in 8.3, this created a perverse incentive: opposing a proposal could help it pass. We reproduced it in a test, then changed the quorum calculation to count For + Abstain only, matching the OpenZeppelin standard. Fixed before the audit scope was frozen.

Finding 2 — Voting power does not decay after the lock expires. Described in 7.4. This is a design consequence rather than a defect: influence can concentrate among early long-term stakers. We accepted and documented it rather than fixing it, because a decay mechanism is a redesign rather than a correction. It remains open to a future governance decision.

10.4 Known limitations

The following are properties of the system that we consider acceptable but which a reader should know. None of them are hidden in the code.

Residual MEV exposure. Automated swaps are protected by a slippage bound, which limits but does not eliminate sandwich attacks. Fully removing the exposure would require a time-weighted price oracle; the bound is the trade-off chosen.

Upgradeability. The token can be upgraded by governance. This is a capability, and capabilities carry risk: a harmful upgrade approved by a majority is possible in principle. The seven-day timelock exists so that such a proposal is visible for a week before it can take effect.

Rounding dust. Reflection and reward calculations use integer division, leaving amounts of a few wei unattributed. We measured this precisely during testnet reward distribution: across three claims, the residue was two wei, each attributable to one floor division. It is accounted for, not ignored.

Router dependency. Automated swaps depend on the PancakeSwap router. If that contract were compromised or the liquidity pool drained, the swap mechanism would degrade. Swaps are wrapped so that failure cannot block ordinary transfers.

The least-proven path. The sequence fee accumulation → swap to BNB → distribution → automatic buyback has

been exercised once on testnet, under laboratory conditions, on a pool with minimal liquidity. It has never run under real slippage, real volume or adversarial conditions, and it cannot be until mainnet. We consider this the least-proven surface of the protocol and have flagged it as such to every auditor we have approached.

10.5 The interface is not the protocol

A question worth answering explicitly: what happens if the website is compromised?

The web interface is a stateless public frontend. It holds no keys, custodies no funds, maintains no database and no authenticated sessions, and never signs anything. Every interaction with the chain happens in the user's own browser, with the user's own wallet, requiring the user's own signature.

The interface is never in the custody path or the signing path. Its attack surface is therefore **availability, not funds** — a compromised or unavailable frontend cannot move anyone's tokens, because it never had the ability to move them in the first place.

The realistic worst case is that the page stops working. The contracts remain fully usable through a block explorer or a command-line tool, as they were throughout months of testing before the interface existed.

This is a deliberate consequence of the architecture rather than a fortunate accident: the protocol is the contracts, and the interface is a convenience placed in front of them. Anything that can be done through the interface can be done without it.

The same reasoning applies to the interface's software dependencies. Known advisories affecting them are documented in `SECURITY.md`, together with an assessment of whether the conditions required to exploit each one exist in this application at all. The contracts themselves have no runtime dependencies of this kind.

10.6 What has been demonstrated on-chain

Beyond automated testing, the following functional cycles have been executed and documented on BSC testnet, each with transaction hashes:

- 1:1 migration, verified to the wei
- fee application and reflection distribution, verified to the wei
- vote-escrow staking with weighted voting power, and rejection of early withdrawal
- the complete governance cycle described in Section 9
- autonomous fee-swap on a real liquidity pool, with BNB actually received by the marketing wallet and the staking reward pool
- the first real burn: 44,785,811 DMN removed from total supply
- multi-wallet reward claims, reconciled to the wei across three addresses
- guardian pause and unpause
- rejection of early execution (`TooEarly`) and of double execution (`AlreadyExecuted`)
- economic verification of the post-execution fee: exactly 4.00%

One further cycle is in progress at the time of writing: the governance proposal that will sweep unclaimed migration tokens to the treasury has been proposed and voted, and awaits its timelock period. It is the last function of the system never yet executed on-chain.

The complete record is published as `TESTNET_RESULTS.md`.

10.7 External audit

Everything above was produced by the same people who wrote the code. That is useful and insufficient.

The contracts are frozen at the tag `audit-scope-v2` and submitted for independent security review. Our commitments regarding it are three:

The report will be published in full, whatever it contains. Not a summary, not a certificate, not selected excerpts.

We will not deploy to mainnet before it is complete. No launch date has been announced, and none will be until the review has concluded.

The auditor receives our own list of concerns, including the least-proven path described in 10.4. An audit exists to find what we missed, not to confirm what we already know.

A public bug bounty is planned following launch, so that the incentive to report a vulnerability permanently exceeds the incentive to exploit it.

11. Funds and treasury

11.1 Two locations, two rules

Protocol funds exist in two places, governed differently, for a reason worth explaining rather than assuming.

The treasury holds the reserve: legacy tokens collected during migration, unclaimed DMN swept after the deadline, and any assets the DAO accumulates. It is controlled by the timelock. Every outflow requires a proposal, a vote, a seven-day delay and an execution. No individual can move a single token from it.

The operational wallet receives the marketing share of transaction fees in BNB. It is a multi-signature wallet with known signers, used for ordinary expenses: services, tooling, design, infrastructure.

11.2 Why not everything under governance

A protocol where every expenditure requires a vote sounds more decentralized. In practice it is unworkable: submitting a proposal, waiting a day, running a five-day vote and waiting seven more days in order to pay for a design asset is not governance, it is paralysis. Projects that adopt it either abandon it quietly or stop spending.

The two-location structure resolves the tension without pretending it does not exist:

- Large decisions — reserves, allocations, anything material — are voted. No trust required.
- Small operational costs are handled by a multisig funded by a limited and ongoing revenue share. Trust required, but **bounded and accountable**.

The operational wallet never holds the protocol's reserves. If a signing key were compromised, the exposure is the current operating balance, not the treasury. The vault stays locked either way.

This is the structure used by mature DAOs — a governed treasury alongside operational multisigs with limited budgets — for the same reasons.

11.3 What governance controls here

Even the operational wallet is not outside the system. The address that receives the marketing share is set by a governance-only function: the DAO can redirect that revenue stream at any time, by vote. What it does not do is approve each individual payment.

Two addresses are permanently outside anyone's reach, including governance: the dead address that receives burned tokens, and the migration treasury. Both are `immutable`, fixed at deployment. Some destinations should not be redirectable by a majority.

11.4 Commitments

The operational wallet's signers will be public. Its spending will be reportable on-chain, because every transaction it makes is visible by construction. And if the community concludes that the arrangement should change — a lower share, a spending cap enforced in code, full treasury control — that is a proposal like any other.

12. Roadmap

This roadmap distinguishes between what exists, what is intended, and what is a long-term possibility. No dates are given beyond the current phase, and nothing below constitutes a commitment to deliver.

It is also, by design, incomplete. Section 12.5 explains why.

12.1 Phase 1 — Now

The protocol described in this document: token, staking, governance, timelock, migration. Deployed and fully exercised on BSC testnet, frozen for external audit, awaiting review before mainnet deployment.

Completing this phase means: audit concluded and published, mainnet deployment, migration window open, initial liquidity in place.

12.2 Phase 2 — Daimon as a DeFi protocol

The current protocol is a token with governance. The intention is for it to become an ecosystem of financial primitives, owned and directed by the people who use it.

The direction is deliberate: the tools that determine what happens to ordinary people's savings — credit, yield, liquidity — are held almost entirely by institutions that set the terms without consultation. Rebuilding them as open-source, permissionless contracts governed by their users is the practical form of the argument made in Section 3.

The primitives we intend to pursue:

Lending and borrowing. Supplying assets to earn interest, and borrowing against collateral. Interest rates determined algorithmically by utilisation rather than by an institution's policy, collateral requirements visible in the code, liquidation rules identical for everyone.

Liquidity provision. Mechanisms for supplying liquidity and earning a share of trading activity, with the terms — fee splits, incentive structure, supported pairs — set by governance rather than by an operator.

Yield strategies. Structured products that route capital into established protocols, with the risk profile of each strategy stated explicitly rather than buried, and the selection of strategies decided by vote.

Further primitives as they mature. Derivatives, structured credit, insurance mechanisms, real-world asset integration, cross-chain liquidity — the DeFi surface expands continuously, and any primitive that can be built as a permissionless contract is a candidate.

How this connects to the token. Each of these generates protocol revenue. That revenue enters the cycle the token already implements: a share to buyback and burn, a share to stakers, a share to the treasury. The protocol's economic engine is already built; Phase 2 is about giving it more to run on.

The stated ambition is that someone with very little capital and no financial training should be able to access the same instruments as anyone else, on identical terms, with the rules visible and the governance open to them. That is the objective. Whether and how far it is reached depends on execution, resources, and time.

Constraints that will apply. Every module will be audited independently before deployment. Every module will be introduced through governance rather than announced. Modules that handle user funds will be built with the same constraints as the core protocol: no owner, no privileged withdrawal, no parameter without a bound.

12.3 Phase 3 — An active treasury

The treasury currently holds assets and does nothing with them. A future capability would allow the DAO to allocate them into approved protocols, so that idle capital generates revenue feeding the same cycle.

Mechanically this is a timelock call to an approved contract, following a vote. The difficulty is not the mechanism: a treasury able to interact with external contracts is the largest and most attractive attack surface a protocol can create. It would require an allowlist of approved protocols rather than arbitrary calls, per-operation limits, a dedicated audit, and deployment only after the core protocol has operated on mainnet long enough to be considered stable.

On what this is not. Capital that generates return is capital that carries risk: third-party contract failure, impermanent loss, market conditions. Returns may be positive, zero, or negative. Nothing here promises yield, and any strategy would be selected by the community with its risks stated explicitly in the proposal.

12.4 Phase 4 — Infrastructure

A distant possibility, included because a roadmap that stops at the comfortable horizon is not a roadmap.

If the ecosystem grew to a point where a general-purpose chain became a genuine constraint — throughput, transaction costs, protocol-level features unavailable on BNB Chain — the community could evaluate moving to dedicated infrastructure.

The realistic form is not building a blockchain from scratch. A new chain starts with no validators and no economic security, which makes it attackable regardless of how well it is engineered. The viable path is an application-specific chain or layer-2 that **inherits** security from an established network rather than attempting to bootstrap its own.

This is explicitly conditional. It happens only if real growth justifies it, and only by community decision.

What would carry over. The tokenomics, the governance model, the staking design, the principle of ownerlessness — these are logic, and logic is portable. A different chain would change where the protocol runs, not what it does.

12.5 The roadmap is not fixed

Everything above will be wrong in some respect, and that is expected.

DeFi does not stand still. Primitives that do not exist today will exist in three years; approaches that look essential now will be superseded; risks not yet identified will emerge. A roadmap written as a fixed sequence of deliverables would be obsolete before the first item shipped — and worse, it would commit the protocol to a plan rather than to a direction.

Daimon is built to change. The upgrade path exists, governance can extend the system, and the constraints that cannot be altered are deliberately few and specifically chosen: no minting, a supply floor, a fee ceiling, a mandatory delay. Everything else is open, because everything else should be able to improve.

This is not a caveat added for safety. It is the concept the protocol was named after.

Heraclitus — whose reading of the daimon appears in Section 2 — is also the philosopher of flux: *everything flows*, nothing remains what it was, and what endures does so precisely by changing. The river persists because the water does not. A system designed to remain identical to itself would not be a faithful implementation of that idea; it would be its opposite.

So the roadmap will move. New frontiers will be added as they appear, and directions taken here will be abandoned when better ones are found. What will not change is the small set of constraints that make the protocol what it is — and the requirement that every change passes through a public vote and seven days in the open.

The destination is not fixed. The method is.

---

13. What Daimon is not

Most whitepapers end with ambition. This one ends with limits, because a reader deciding whether to participate is better served by knowing what can go wrong than by another paragraph on what could go right.

Daimon is not an investment product and does not promise returns. There is no yield target, no projection, no expectation of appreciation stated anywhere in this document. The protocol guarantees rules — a supply that cannot be inflated, parameters that cannot be changed in private, a floor that cannot be crossed. It guarantees nothing about price, which is determined by a market that no code controls.

Deflation is not a price mechanism. A shrinking supply does not mechanically increase value. Demand can fall faster than supply. The floor guarantees scarcity; it guarantees nothing else.

There is no team that will protect you. This is the point of the design, and it cuts both ways. No one can change the rules against you, and no one can intervene to help you either. There is no support desk that can reverse a transaction, recover a lost key, or compensate a loss. Ownerlessness means exactly what it says.

Governance can make mistakes. A majority can approve a bad proposal. The constraints in the code bound the damage — fees cannot exceed 10%, supply cannot be minted or burned below the floor, funds cannot be redirected to arbitrary addresses — but within those bounds, the community can decide poorly. The seven-day timelock exists so that a mistake is visible before it takes effect, not so that it becomes impossible.

The code may contain flaws. It has been tested extensively, analysed statically, attacked deliberately by its own authors, and submitted for independent review. None of that makes it perfect. Contracts audited by the best firms in the industry have been exploited. The honest statement is that we have reduced the probability of failure as far as we know how, not that we have eliminated it.

Participation is voluntary and carries risk. Tokens can lose value. Locked tokens cannot be withdrawn early. Smart contracts can fail. Nothing here should be understood as advice to acquire, hold, or stake anything.

We are not neutral. This document was written by the people who built the protocol. We believe in it, which is a reason to read our claims critically rather than accept them. Everything asserted here corresponds to code that is public and deployed. The appropriate response to this document is not to trust it. It is to verify it.

---

14. Legal disclaimer

This document is provided for informational purposes only. It does not constitute an offer to sell, a solicitation to buy, or a recommendation regarding any digital asset, security, or financial instrument.

Nothing in this document constitutes investment, financial, legal, tax, or accounting advice. Readers should conduct their own research and consult qualified professionals before making any decision.

Digital assets involve substantial risk, including the total loss of the amount involved. Past performance, testnet results, and technical characteristics do not indicate or guarantee future outcomes. Smart contracts may contain vulnerabilities despite testing and independent review.

Daimon is a decentralized protocol with no owner, no administrator, and no legal entity operating it. No party guarantees its continued operation, maintains an obligation to support it, or is able to reverse, modify, or compensate transactions executed by its contracts.

Statements in this document regarding future development represent intentions and possibilities, not commitments. Any future phase depends on community governance decisions, technical feasibility, and available resources.

The regulatory treatment of digital assets varies by jurisdiction and continues to evolve. Readers are responsible for determining whether their participation is lawful where they reside.

The information in this document is accurate to the best of the authors' knowledge at the date of publication. Contract addresses, parameters, and technical details should be verified directly on-chain, which remains the authoritative source in all cases.

Repository: `github.com/daimon-dao/daimon-dao` **Audit scope:** tag `audit-scope-v2`

Draft v0.1 — pending external audit. This document will be updated to reflect the audit outcome before publication.